

DAA Lab 1:

Part 1: Understanding Time and Space Complexity

When we design algorithms, there are often multiple ways to solve the same problem. To determine which solution is "best," we evaluate algorithms based on their efficiency. This is where Time and Space Complexity come into play.

1. Time Complexity

Time complexity is not the actual time in seconds that a machine takes to execute code, because execution time varies depending on hardware configuration, background processes, and compiler optimizations.

Instead, Time Complexity measures the **growth rate of the number of operations** an algorithm performs with respect to the input size (N). It helps us understand how the algorithm's performance scales as the data gets larger.

Key Asymptotic Notations:

- **Big O (O):** Represents the upper bound or the worst-case scenario. This guarantees that the algorithm will take no longer than a specific amount of time. (Most commonly used in interviews and practicals).
- **Omega (Ω):** Represents the lower bound or the best-case scenario.
- **Theta (Θ):** Represents the exact bound or average-case scenario.

Common Time Complexities:

- **O(1) - Constant Time:** The operation count does not change regardless of input size (e.g., accessing an array element by index).
- **O(N) - Linear Time:** The operation count scales directly in proportion to the input size (e.g., looping through an array once).
- **O(N²) - Quadratic Time:** The operation count is proportional to the square of the input size (e.g., nested loops).
- **O(log N) - Logarithmic Time:** The operation count increases very slowly, typically by halving the search space in each step (e.g., Binary Search).

2. Space Complexity

Space Complexity measures the total amount of memory an algorithm utilizes relative to the input size.

Space Complexity consists of two parts:

1. **Input Space:** The memory required to store the initial inputs provided to the algorithm.
2. **Auxiliary Space:** The extra or temporary memory used by the algorithm during its execution (e.g., temporary variables, new arrays, or the recursive call stack).

When we talk about optimizing an algorithm's space complexity, we are usually trying to minimize the **Auxiliary Space**.

Part 2: Practical 1 - Check Whether a Number is Prime or Not

A prime number is a natural number greater than 1 that has exactly two distinct positive divisors: 1 and itself. Below are 4 progressive solutions to solve this problem, starting from the naive approach to the most optimized approach.

Solution 1: The Naive Approach

The simplest way is to iterate from 2 up to $N-1$ and check if any number divides N evenly.

- **Time Complexity:** $O(N)$
- **Space Complexity:** $O(1)$

```
public class PrimeSolution1 {
    public static boolean isPrime(int n) {
        if (n <= 1) return false;

        for (int i = 2; i < n; i++) {
            if (n % i == 0) {
                return false;
            }
        }

        return true;
    }
}
```

Solution 2: Iterate up to $N/2$

The largest possible factor of a number N (excluding N itself) can never be greater than $N/2$. Therefore, we can reduce our loop to run only up to half of N .

- **Time Complexity:** $O(N)$
- **Space Complexity:** $O(1)$

```
public class PrimeSolution2 {
    public static boolean isPrime(int n) {
        if (n <= 1) return false;

        for (int i = 2; i <= n / 2; i++) {
            if (n % i == 0) {
```

```

        return false;
    }
}

return true;
}
}

```

Solution 3: Iterate up to the Square Root of N

Mathematically, if a number N has factors, at least one of those factors will be less than or equal to the square root of N . Thus, we only need to iterate up to $\sqrt{N}$. This significantly optimizes the algorithm.

- **Time Complexity:** $O(\sqrt{N})$
- **Space Complexity:** $O(1)$

```

public class PrimeSolution3 {
    public static boolean isPrime(int n) {
        if (n <= 1) return false;

        for (int i = 2; i * i <= n; i++) {
            if (n % i == 0) {
                return false;
            }
        }

        return true;
    }
}

```

Solution 4: The Most Optimized Approach (Skipping Multiples of 2 and 3)

All prime numbers greater than 3 can be expressed in the form $6k \pm 1$. We can handle 2 and 3 as edge cases, skip all their multiples, and then only check remaining numbers by jumping 6 steps at a time.

- **Time Complexity:** $O(\sqrt{N})$
- **Space Complexity:** $O(1)$

```

public class PrimeSolution4 {
    public static boolean isPrime(int n) {
        if (n <= 1) return false;

```

```

    if (n == 2 || n == 3) return true;
    if (n % 2 == 0 || n % 3 == 0) return false;

    for (int i = 5; i * i <= n; i += 6) {
        if (n % i == 0 || n % (i + 2) == 0) {
            return false;
        }
    }

    return true;
}

```

Part 3: Practical 2 - Search Insert Position

Problem Statement: Given a sorted array of distinct integers and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order.

Solution 1: Linear Search (Naive Approach)

In this approach, we iterate through the array from the beginning. Since the array is sorted, the first element we encounter that is greater than or equal to the target will give us the correct insert position.

- **Time Complexity:** $O(N)$ where N is the number of elements in the array.
- **Space Complexity:** $O(1)$

```

public class SearchInsertLinear {
    public static int searchInsert(int[] nums, int target) {
        for (int i = 0; i < nums.length; i++) {
            if (nums[i] >= target) {
                return i;
            }
        }
        return nums.length;
    }
}

```

Solution 2: Binary Search (Optimized Approach)

Since the given array is sorted, we can optimize the search by using Binary Search. We maintain a search space defined by a left and right pointer and repeatedly halve it until we find the target or the pointers cross. When they cross, the left pointer will indicate the exact index where the

target should be inserted.

- **Time Complexity:** $O(\log N)$ which is highly efficient for large datasets.
- **Space Complexity:** $O(1)$

```
public class SearchInsertBinary {  
    public static int searchInsert(int[] nums, int target) {  
        int left = 0;  
        int right = nums.length - 1;  
  
        while (left <= right) {  
            int mid = left + (right - left) / 2;  
  
            if (nums[mid] == target) {  
                return mid;  
            } else if (nums[mid] < target) {  
                left = mid + 1;  
            } else {  
                right = mid - 1;  
            }  
        }  
  
        return left;  
    }  
}
```